



EXVUL

SMART CONTRACT **AUDIT REPORT**

AquaFlux Smart Contract

JANUARY 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Calculation error of addReward led to incorrect allocation	9
3.2 calculation error of _unwrap caused the assetUnderlyingBalances statistic too high . . .	12
3.3 resetDistribution Allows Re-Planning After Claims Causing Over Distribution	16
3.4 updateDelay() bypasses the MINIMUM_ALLOWED_MIN_DELAY restriction	20
3.5 Missing Token Management Interface in AquaFluxCore Leads to Loss of Governance . . .	22
3.6 delay setting in emergencyExecute() does not match the documentation	25
3.7 register Bypasses Global Pause Mechanism causing Incomplete Control	27
3.8 updateAssetMetadata Allows Post Maturity Parameter Modifications	29
3.9 getAssetLifecycleState Skips States and Provides False Claimable Status	31
3.10 address(0) allows being added to the blacklist	33
3.11 Comments do not match code logic	35
4. CONCLUSION	37
5. APPENDIX	38
5.1 Basic Coding Assessment	38
5.1.1 Apply Verification Control	38
5.1.2 Authorization Access Control	38
5.1.3 Forged Transfer Vulnerability	38
5.1.4 Transaction Rollback Attack	39
5.1.5 Transaction Block Stuffing Attack	39
5.1.6 Soft Fail Attack Assessment	39
5.1.7 Hard Fail Attack Assessment	40
5.1.8 Abnormal Memo Assessment	40
5.1.9 Abnormal Resource Consumption	40
5.1.10 Random Number Security	41
5.2 Advanced Code Scrutiny	41
5.2.1 Cryptography Security	41

5.2.2 Account Permission Control	41
5.2.3 Malicious Code Behavior	42
5.2.4 Sensitive Information Disclosure	42
5.2.5 System API	42
6. DISCLAIMER	43
7. REFERENCES	44
8. About Exvul Security	45

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **AquaFlux** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
AquaFlux	15/01/2026 - 03/02/2026	Solidity

Repository

Commit Hash

813d88136953c9e3cb2d4984297b7599

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	2
MEDIUM	3
LOW	5
INFO	1



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Calculation error of addReward led to incorrect allocation	Fixed
HIGH	calculation error of _unwrap caused the assetUnderlyingBalances statistic too high	Fixed
MEDIUM	resetDistribution Allows Re-Planning After Claims Causing Over Distribution	Fixed
MEDIUM	updateDelay() bypasses the MINIMUM_ALLOWED_MIN_DELAY restriction	Fixed
MEDIUM	Missing Token Management Interface in AquaFluxCore Leads to Loss of Governance	Fixed
LOW	delay setting in emergencyExecute() does not match the documentation	Fixed
LOW	register Bypasses Global Pause Mechanism causing Incomplete Control	Fixed
LOW	updateAssetMetadata Allows Post Maturity Parameter Modifications	Fixed
LOW	getAssetLifecycleState Skips States and Provides False Claimable Status	Fixed
LOW	address(0) allows being added to the blacklist	Fixed
INFO	Comments do not match code logic	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Calculation error of addReward led to incorrect allocation

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`contracts/staking/MaturityStaking.sol`**DESCRIPTION:**

The calculation logic of the `addReward` function uses `totalRewardAdded` - `totalRewardClaimed` to calculate remaining distributable rewards. However, `totalRewardClaimed` only represents rewards already withdrawn and does not include rewards allocated but not yet withdrawn.

```
function addReward(uint256 amount) external onlyOwner beforeMaturity {
    require(amount > 0, "Cannot add 0 reward");

    // Update state *before* changing parameters
    _updateReward();

    rewardToken.safeTransferFrom(msg.sender, address(this), amount);

    totalRewardAdded += amount;

    // Recalculate reward rate based on remaining time and remaining
    rewards
    uint256 remainingTime = poolEndTime - lastUpdateTime;
    uint256 remainingReward = (totalRewardAdded > totalRewardClaimed)
        ? totalRewardAdded - totalRewardClaimed
        : 0;

    // Prevent division by zero when pool is about to end
    if (remainingTime > 0) {
        rewardPerSecond = remainingReward / remainingTime;
    } else {
        rewardPerSecond = 0;
    }
}
```

```
        emit RewardAdded(amount, remainingTime);
    }
```

IMPACT:

The flawed calculation logic can create an excessively large `rewardPerSecond`, causing early stakers after reward addition to receive more rewards while later users receive less, and causing reward exhaustion earlier than intended.

RECOMMENDATIONS:

Replace the remaining reward calculation with accrued reward tracking to avoid overstating available rewards.

```
function addReward(uint256 amount) external onlyOwner beforeMaturity {
    require(amount > 0, "Cannot add 0 reward");

    // Update state *before* changing parameters
    _updateReward();

    rewardToken.safeTransferFrom(msg.sender, address(this), amount);

    totalRewardAdded += amount;

    // Recalculate reward rate based on remaining time and remaining
    rewards
    uint256 remainingTime = poolEndTime - lastUpdateTime;
-   uint256 remainingReward = (totalRewardAdded > totalRewardClaimed)
-       ? totalRewardAdded - totalRewardClaimed
-       : 0;
+   uint256 remainingReward = (totalRewardAdded > totalAccruedRewards)
+       ? totalRewardAdded - totalAccruedRewards
+       : 0;
    // Prevent division by zero when pool is about to end
    if (remainingTime > 0) {
        rewardPerSecond = remainingReward / remainingTime;
    } else {
        rewardPerSecond = 0;
    }

    emit RewardAdded(amount, remainingTime);
}
```

}

3.2 calculation error of `_unwrap` caused the `assetUnderlyingBalances` statistic too high

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

contracts/libraries/OperationsLib.sol

DESCRIPTION:

In the `_unwrap` accounting logic, only `netAmount` is deducted from `assetUnderlyingBalances`, while `feeAmount` remains in tracked underlying balance. This causes `assetUnderlyingBalances` to be overstated.

```
function _unwrap(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => uint256) storage assetUnderlyingBalances,
    mapping(bytes32 => mapping(IAquaFluxCore.OperationType =>
uint16)) storage assetFeeRates,
    mapping(bytes32 => uint256) storage assetFeesTotal,
    mapping(bytes32 => mapping(string => uint256)) storage
assetFeesByOperation,
    mapping(bytes32 => uint256) storage assetFeesWithdrawable,
    bytes32 assetId,
    uint256 amount,
    address user
) internal returns (uint256 netAmount, uint256 feeAmount) {
    if (amount == 0) revert AmountMustBeGreaterThanZero();

    IAquaFluxCore.AssetInfo storage asset = assets[assetId];
    if (asset.aqToken == address(0)) revert AqTokenNotDeployed();

    // Collect fee
    feeAmount = _collectFee(
        assetFeeRates,
        assetFeesTotal,
        assetFeesByOperation,
        assetFeesWithdrawable,
        IAquaFluxCore.OperationType.UNWRAP,
        assetId,
```

```
        amount
    );
    netAmount = amount - feeAmount;

    // Burn AqTokens from user
    IBaseToken(asset.aqToken).burn(user, amount);

    // Update asset-specific underlying balance tracking
    if (assetUnderlyingBalances[assetId] < netAmount) revert
    InsufficientContractBalance();
    assetUnderlyingBalances[assetId] -= netAmount;

    // Transfer underlying tokens to user (net amount after fee)
    IERC20(asset.underlying).safeTransfer(user, netAmount);

    return (netAmount, feeAmount);
}
```

An overstated `assetUnderlyingBalances` can later break protocol fee withdrawal safety checks.

```
function withdrawProtocolFees(
    bytes32 assetId,
    address to
) external onlyRole(TIMELOCK_ROLE) nonReentrant {
    if (!isAssetRegistered(assetId)) revert AssetNotRegistered();
    if (to == address(0)) revert InvalidRecipientAddress();

    uint256 amount = assetFeesWithdrawable[assetId];
    if (amount == 0) revert AmountMustBeGreaterThanZero();

    AssetInfo storage asset = assets[assetId];
    IERC20 underlying = IERC20(asset.underlying);

    // Check contract has sufficient balance
    uint256 contractBalance = underlying.balanceOf(address(this));
    if (contractBalance < amount) revert
    InsufficientContractBalance();

    // Additional safety check: verify total tracked balances don't
    exceed contract balance
    // This helps detect accounting errors that could lead to
```

```
withdrawing wrong funds
    uint256 trackedAssetBalance = assetUnderlyingBalances[assetId] +
assetFeesWithdrawable[assetId];
    if (trackedAssetBalance > contractBalance) {
        // This indicates accounting error - tracked balances exceed
actual contract balance
        revert InsufficientContractBalance();
    }

    // Transfer all fees to recipient
    underlying.safeTransfer(to, amount);

    // Clear the withdrawable fee balance after successful transfer
    assetFeesWithdrawable[assetId] = 0;

    emit ProtocolFeesWithdrawn(assetId, to, amount, msg.sender);
}
```

IMPACT:

This accounting mismatch can block [withdrawProtocolFees](#) because tracked balances exceed actual contract balance, which can freeze protocol fee withdrawals.

```
// Assuming fee is 10%
user wrap 100rwa
balance: 100 , assetUnderlyingBalances: 90 , assetFeesWithdrawable: 10
user unwrap 90rwa
balance: 19 , assetUnderlyingBalances: 9 , assetFeesWithdrawable: 19
// now
balance < assetUnderlyingBalances + assetFeesWithdrawable
```

RECOMMENDATIONS:

Deduct the full [amount](#) from [assetUnderlyingBalances](#) to keep accounting consistent with token burn and fee accrual.

```
function _unwrap(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => uint256) storage assetUnderlyingBalances,
    mapping(bytes32 => mapping(IAquaFluxCore.OperationType =>
```

```
uint16)) storage assetFeeRates,
    mapping(bytes32 => uint256) storage assetFeesTotal,
    mapping(bytes32 => mapping(string => uint256)) storage
assetFeesByOperation,
    mapping(bytes32 => uint256) storage assetFeesWithdrawable,
    bytes32 assetId,
    uint256 amount,
    address user
) internal returns (uint256 netAmount, uint256 feeAmount) {
    if (amount == 0) revert AmountMustBeGreaterThanZero();

    IAquaFluxCore.AssetInfo storage asset = assets[assetId];
    if (asset.aqToken == address(0)) revert AqTokenNotDeployed();

    // Collect fee
    feeAmount = _collectFee(
        assetFeeRates,
        assetFeesTotal,
        assetFeesByOperation,
        assetFeesWithdrawable,
        IAquaFluxCore.OperationType.UNWRAP,
        assetId,
        amount
    );
    netAmount = amount - feeAmount;

    // Burn AqTokens from user
    IBaseToken(asset.aqToken).burn(user, amount);

    // Update asset-specific underlying balance tracking
-     if (assetUnderlyingBalances[assetId] < netAmount) revert
InsufficientContractBalance();
-     assetUnderlyingBalances[assetId] -= netAmount;
+     if (assetUnderlyingBalances[assetId] < amount) revert
InsufficientContractBalance();
+     assetUnderlyingBalances[assetId] -= amount;
    // Transfer underlying tokens to user (net amount after fee)
    IERC20(asset.underlying).safeTransfer(user, netAmount);

    return (netAmount, feeAmount);
}
```

3.3 resetDistribution Allows Re-Planning After Claims Causing Over Distribution

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/libraries/MaturityLib.sol

DESCRIPTION:

`setDistributionPlan` snapshots current AQ/P/C/S/SS total supplies into fixed supply fields, and `claimMaturityReward` uses those fixed supplies for proportional reward allocation. Because claim burns user tokens, a later reset-and-replan can snapshot smaller supplies and create over-distribution.

```
function setDistributionPlan(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => IAquaFluxCore.AssetMaturityManagement) storage
    maturityManagement,
    bytes32 assetId
) external returns (
    uint256 aqAllocation,
    uint256 pAllocation,
    uint256 cAllocation,
    uint256 sAllocation,
    uint256 ssAllocation
) {
    // ...
    // Snapshot AQ/P/C/S/SS supply (for user claim ratio calculation)
    uint256 aqSupply = asset.aqToken != address(0) ?
IERC20(asset.aqToken).totalSupply() : 0;
    uint256 pSupply = asset.pToken != address(0) ?
IERC20(asset.pToken).totalSupply() : 0;
    uint256 cSupply = asset.cToken != address(0) ?
IERC20(asset.cToken).totalSupply() : 0;
    uint256 sSupply = asset.sToken != address(0) ?
IERC20(asset.sToken).totalSupply() : 0;
    uint256 ssSupply = asset.stakedS != address(0) ?
IERC20(asset.stakedS).totalSupply() : 0;
}
```



```
function claimMaturityReward(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => IAquaFluxCore.AssetMaturityManagement) storage
    maturityManagement,
    mapping(bytes32 => mapping(address => uint256)) storage
    claimFeesCollected,
    mapping(bytes32 => mapping(IAquaFluxCore.OperationType =>
    uint16)) storage assetFeeRates,
    address tokenAddress,
    address user
) external returns (uint256 userBalance, uint256 netReward, uint256
feeAmount) {
    // ...
    uint256 userReward = Math.mulDiv(tokenAllocation, userBalance,
    fixedTotalSupply);
    if (userReward == 0) revert NoRewardToClaim();

    // Calculate claim fee
    feeAmount = 0;
    netReward = userReward;
    uint16 claimFeeRate =
    assetFeeRates[assetId][IAquaFluxCore.OperationType.CLAIM];
    if (claimFeeRate > 0) {
        feeAmount = Math.mulDiv(userReward, claimFeeRate, 10000);
        netReward = userReward - feeAmount;
    }

    // Get distribution config
    IAquaFluxCore.DistributionConfig storage config = maturity.config;
    address distributionAddress = config.distributionAddress;

    // Check distribution address has sufficient balance and allowance
    IERC20 distributionToken = IERC20(config.distributionToken);
    uint256 distributionBalance =
    distributionToken.balanceOf(distributionAddress);
    uint256 allowance =
    distributionToken.allowance(distributionAddress, address(this));

    if (distributionBalance < userReward) revert
    InsufficientDistributionBalance();
    if (allowance < userReward) revert InsufficientAllowance();
```

```
// Burn the user's tokens to prevent re-claiming
IBaseToken(tokenAddress).burn(user, userBalance);
}
```

`resetDistribution` can currently be called by `TIMELOCK_ROLE` without checking whether claims already started, and it clears plan/config state.

```
function resetDistribution(
    mapping(bytes32 => IAquaFluxCore.AssetMaturityManagement) storage
    maturityManagement,
    bytes32 assetId
) external {
    IAquaFluxCore.AssetMaturityManagement storage maturity =
    maturityManagement[assetId];
    if (!maturity.isDistributionConfigured) revert
    DistributionConfigNotSet();

    delete maturity.config;
    delete maturity.plan;
    maturity.isDistributionConfigured = false;
    maturity.isDistributionPlanned = false;
    maturity.openingNavBps = 0;
    maturity.closingNavBps = 0;
}
```

After reset, `setDistributionPlan` can be called again because `isDistributionPlanned` is false.

```
function setDistributionPlan(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => IAquaFluxCore.AssetMaturityManagement) storage
    maturityManagement,
    bytes32 assetId
) external returns (
    uint256 aqAllocation,
    uint256 pAllocation,
    uint256 cAllocation,
    uint256 sAllocation,
    uint256 ssAllocation
) {
    // ...
    if (maturity.isDistributionPlanned)
```

```
        revert DistributionAlreadySet();
    // ...
    maturity.isDistributionPlanned = true;
}
```

IMPACT:

This can lead to over-distribution to one tranche and underfunding for another tranche. In the demonstrated flow, P holders can receive 150 percent of intended allocation, leaving insufficient balance for S holders and causing `InsufficientDistributionBalance()` reverts for later claims.

RECOMMENDATIONS:

Forbid reset once any claim has started, and mark claim start in `claimMaturityReward`.

```
function resetDistribution(bytes32 assetId) external {
    IAquaFluxCore.AssetMaturityManagement storage maturity =
        maturityManagement[assetId];
+   if (maturity.claimsStarted) revert CannotResetAfterClaims();
    delete maturity.config;
    delete maturity.plan;
    // ...
}

function claimMaturityReward(...) external returns (...) {
    // ...
+   if (!maturity.claimsStarted) maturity.claimsStarted = true;
    IBaseToken(tokenAddress).burn(user, userBalance);
    // ...
}
```

3.4 updateDelay() bypasses the MINIMUM_ALLOWED_MIN_DELAY restriction

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`contracts/governance/AquaFluxTimelock.sol`**DESCRIPTION:**

The constructor enforces `MINIMUM_ALLOWED_MIN_DELAY`, but inherited `updateDelay()` from OpenZeppelin `TimelockController` can later set `_minDelay` below this project-specific minimum unless overridden.

```
// contracts/governance/AquaFluxTimelock.sol
    constructor(
        uint256 minDelay,
        address[] memory proposers,
        address[] memory executors,
        address admin
    ) TimelockController(minDelay, proposers, executors, admin) {
        require(
            minDelay >= MINIMUM_ALLOWED_MIN_DELAY,
            "minDelay below allowed minimum"
        );
        _configureDefaults();
    }
// openzeppelin/contracts/governance/TimelockController.sol
    function updateDelay(uint256 newDelay) public virtual {
        address sender = _msgSender();
        if (sender != address(this)) {
            revert TimelockUnauthorizedCaller(sender);
        }
        emit MinDelayChange(_minDelay, newDelay);
        _minDelay = newDelay;
    }
```

IMPACT:

A scheduled governance operation can reduce delay below `MINIMUM_ALLOWED_MIN_DELAY`, violating expected governance safety settings.

RECOMMENDATIONS:

Override `updateDelay()` and enforce the same minimum constraint used by the constructor.

```
+   function updateDelay(uint256 newDelay) public virtual override {  
+       require(  
+           newDelay >= MINIMUM_ALLOWED_MIN_DELAY,  
+           "newDelay below allowed minimum"  
+       );  
+       super.updateDelay(newDelay);  
+   }
```

3.5 Missing Token Management Interface in AquaFluxCore Leads to Loss of Governance

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/core/TokenFactory.sol \ contracts/token/BaseToken.sol \ contracts/core/AquaFluxCore.sol

DESCRIPTION:

[AquaFluxCore](#) deploys child tokens through [TokenFactory](#), so the factory sees [AquaFluxCore](#) as caller and grants privileged roles to [AquaFluxCore](#). However, [AquaFluxCore](#) does not expose management interfaces for those child-token controls.

```
// contracts/core/TokenFactory.sol
function deployToken(
    string calldata tokenType,
    bytes32 assetId,
    string calldata name,
    string calldata symbol
) external override onlyRole(DEPLOYER_ROLE) returns (address) {
    // ...
    if (keccak256(abi.encodePacked(tokenType)) ==
keccak256(abi.encodePacked("AQ"))) {
        AqToken(clone).initialize(name, symbol, assetId, msg.sender);
    } else if (keccak256(abi.encodePacked(tokenType)) ==
keccak256(abi.encodePacked("P"))) {
        PToken(clone).initialize(name, symbol, assetId, msg.sender);
    } else if (keccak256(abi.encodePacked(tokenType)) ==
keccak256(abi.encodePacked("C"))) {
        CToken(clone).initialize(name, symbol, assetId, msg.sender);
    } else if (keccak256(abi.encodePacked(tokenType)) ==
keccak256(abi.encodePacked("S"))) {
        SToken(clone).initialize(name, symbol, assetId, msg.sender);
    } else {
        revert("Unsupported token type");
    }
}
```

```
// contracts/token/BaseToken.sol
contract BaseToken is
    Initializable,
    ERC20Upgradeable,
    ERC20BurnableUpgradeable,
    PausableUpgradeable,
    AccessControlUpgradeable,
    IBaseToken
{
    // ...
    /**
     * @dev Pauses all token transfers
     */
    function pause() public override onlyRole(REGISTRY_ROLE) {
        _pause();
    }

    /**
     * @dev Unpauses all token transfers
     */
    function unpause() public override onlyRole(REGISTRY_ROLE) {
        _unpause();
    }

    /**
     * @dev Hook that is called before any transfer of tokens
     (OpenZeppelin 5.x use _update)
     */
    function _update(address from, address to, uint256 value) internal
    virtual override(ERC20Upgradeable) {
        _requireNotPaused();

        // Check if sender or recipient is blocked
        require(!blockedAddresses[from], "BaseToken: sender is blocked");
        require(!blockedAddresses[to], "BaseToken: recipient is blocked");

        super._update(from, to, value);
    }
}
```

IMPACT:

Although [AquaFluxCore](#) holds privileged roles for child tokens, governance cannot trigger pause/un-pause or related management flows through core interfaces, creating operational control gaps during incidents.

RECOMMENDATIONS:

Add management entry points in [AquaFluxCore](#) for authorized operators.

```
// contracts/core/AquaFluxCore.sol
+   function pauseToken(address token) external onlyRole(OPERATOR_ROLE) {
+       IBaseToken(token).pause();
+   }
+
+   function unpauseToken(address token) external onlyRole(OPERATOR_ROLE)
+   {
+       IBaseToken(token).unpause();
+   }
```


3.6 delay setting in emergencyExecute() does not match the documentation

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/governance/AquaFluxTimelock.sol

DESCRIPTION:

`emergencyExecute()` uses `getMinDelay()` as emergency delay, while documentation indicates emergency path should use the fastest allowed delay (`MINIMUM_ALLOWED_MIN_DELAY`).

```
// contracts/governance/AquaFluxTimelock.sol
function emergencyExecute(
    address target,
    uint256 value,
    bytes calldata data,
    string calldata reason
) external returns (bytes32) {
    // ...

    // Use global minDelay for emergency operations (fastest allowed)
    uint256 delay = getMinDelay();

    // ...
}
```

IMPACT:

Emergency operations can be delayed longer than intended by design documentation, reducing responsiveness during urgent scenarios.

RECOMMENDATIONS:

Use `MINIMUM_ALLOWED_MIN_DELAY` explicitly for emergency execution delay.

```
function emergencyExecute(
```

```
        address target,
        uint256 value,
        bytes calldata data,
        string calldata reason
    ) external returns (bytes32) {
        // ...

-        // Use global minDelay for emergency operations (fastest allowed)
-        uint256 delay = getMinDelay();
+        // Use MINIMUM_ALLOWED_MIN_DELAY for emergency operations
+        (fastest allowed)
+        uint256 delay = MINIMUM_ALLOWED_MIN_DELAY;
        // ...
    }
```

3.7 register Bypasses Global Pause Mechanism causing Incomplete Control

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/AquaFluxCore.sol

DESCRIPTION:

`register` performs protocol operations including registration and fund-related setup, but unlike `wrap`, `split`, `combine`, `unwrap`, and `claimMaturityReward`, it is missing `whenNotPaused`.

```
// docs/PROTOCOL_DESIGN.md
Layer 3: Operational Security

Runtime protection mechanisms:
- Global pause: Halts all protocol operations
- Asset-level pause: Halts specific asset operations
- Emergency withdrawal: Timelock-protected fee extraction
- Blocklist functionality: Compliance and sanctions enforcement
// contracts/core/AquaFluxCore.sol
function register(
    AssetType assetType,
    address underlying,
    uint256 maturity,
    uint256 couponRate,
    uint256 principalToS,
    uint256 couponToC,
    uint256 couponToS,
    uint256 protocolFeesToS,
    uint256 couponStartTime,
    string calldata name,
    string calldata metadataURI
) external override returns (bytes32 assetId) {
```

IMPACT:

When protocol pause is active, `register` can still execute, creating mismatch with documented global pause guarantees.

RECOMMENDATIONS:

Apply `whenNotPaused` on `register`.

```
function register(  
    AssetType assetType,  
    address underlying,  
    uint256 maturity,  
    uint256 couponRate,  
    uint256 principalToS,  
    uint256 couponToC,  
    uint256 couponToS,  
    uint256 protocolFeesToS,  
    uint256 couponStartTime,  
    string calldata name,  
    string calldata metadataURI  
-   ) external override returns (bytes32 assetId) {  
+   ) external override whenNotPaused returns (bytes32 assetId) {
```

3.8 updateAssetMetadata Allows Post Maturity Parameter Modifications

SEVERITY: LOW**STATUS:** Fixed**PATH:**`contracts/core/AquaFluxCore.sol`**DESCRIPTION:**

`updateAssetMetadata` allows `TIMELOCK_ROLE` to modify key distribution parameters even after maturity.

```
// contracts/core/AquaFluxCore.sol
function updateAssetMetadata(
    bytes32 assetId,
    uint256 newPrincipalToS,
    uint256 newCouponToC,
    uint256 newCouponToS,
    uint256 newProtocolFeesToS,
    string calldata newMetadataURI
) external onlyRole(TIMELOCK_ROLE) {
    // ...
    asset.principalToS = newPrincipalToS;
    asset.couponToC = newCouponToC;
    asset.couponToS = newCouponToS;
    asset.protocolFeesToS = newProtocolFeesToS;
    asset.metadataURI = newMetadataURI;
    // ...
}
```

IMPACT:

Post-maturity parameter changes introduce centralization risk and can alter payout distribution expectations after user funds are effectively locked for redemption.

RECOMMENDATIONS:

Block distribution-parameter updates after maturity.

```
function updateAssetMetadata(  
    bytes32 assetId,  
    uint256 newPrincipalToS,  
    uint256 newCouponToC,  
    uint256 newCouponToS,  
    uint256 newProtocolFeesToS,  
    string calldata newMetadataURI  
) external onlyRole(TIMELOCK_ROLE) {  
    // ...  
+     if (block.timestamp >= assets[assetId].maturity) {  
+         revert("Cannot modify after maturity");  
+     }  
    // ...  
}
```

3.9 getAssetLifecycleState Skips States and Provides False Claimable Status

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/AquaFluxCore.sol

DESCRIPTION:

`getAssetLifecycleState` skips `DISTRIBUTION_SET` and reports `CLAIMABLE` based only on `isDistributionPlanned`, without checking if claim operation is paused.

```
// contracts/core/AquaFluxCore.sol
// Asset lifecycle states
enum AssetLifecycleState {
    ACTIVE, // Active trading state
    OPERATIONS_STOPPED, // Operations stopped (reached maturity)
    FUNDS_WITHDRAWN, // Funds withdrawn for offline redemption
    DISTRIBUTION_CONFIG_SET, // Distribution config set
    DISTRIBUTION_SET, // Distribution plan set
    CLAIMABLE // Users can claim rewards
}

function getAssetLifecycleState(bytes32 assetId) external view
returns (uint8) {
    // ...
    if (!management.isDistributionPlanned) {
        return uint8(AssetLifecycleState.DISTRIBUTION_CONFIG_SET); //
Returns 3
    }

    return uint8(AssetLifecycleState.CLAIMABLE); // Returns 5,
    skipping 4
}
```

IMPACT:

The function can report an asset as claimable while claim operations are paused, causing misleading

state reporting and confusing user experience.

RECOMMENDATIONS:

Return `DISTRIBUTION_SET` when claim pause is active, and return `CLAIMABLE` only when claims are actually enabled.

```
function getAssetLifecycleState(bytes32 assetId) external view
returns (uint8) {
    // ...
    if (!management.isDistributionPlanned) {
        return uint8(AssetLifecycleState.DISTRIBUTION_CONFIG_SET);
    }

+   if (isOperationPaused(assetId, PAUSE_CLAIM)) {
+       return uint8(AssetLifecycleState.DISTRIBUTION_SET);
+   }

    return uint8(AssetLifecycleState.CLAIMABLE);
}
```


3.10 address(0) allows being added to the blacklist

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/token/BaseToken.sol

DESCRIPTION:

`addToBlocklist` does not validate input addresses, so `address(0)` can be blocked.

```
// contracts/token/BaseToken.sol
function addToBlocklist(
    address[] calldata accounts
) external onlyRole(REGISTRY_ROLE) {
    for (uint256 i; i < accounts.length; ++i) {
        blockedAddresses[accounts[i]] = true;
    }
    emit BlockedAddressesAdded(accounts);
}
```

IMPACT:

If `address(0)` is blocklisted, mint and burn operations can unexpectedly revert, disrupting token functionality.

RECOMMENDATIONS:

Reject zero-address entries in blacklist input.

```
function addToBlocklist(
    address[] calldata accounts
) external onlyRole(REGISTRY_ROLE) {
    for (uint256 i; i < accounts.length; ++i) {
+         require(accounts[i] != address(0), "BaseToken: Invalid
Account")
        blockedAddresses[accounts[i]] = true;
    }
}
```

```
    }  
    emit BlockedAddressesAdded(accounts);  
}
```

3.11 Comments do not match code logic

SEVERITY: INFO**STATUS:** Fixed**PATH:**

contracts/staking/MaturityStaking.sol

DESCRIPTION:

The comment in `_getPendingReward` does not match the formula variable naming used by the implementation.

```
// contracts/staking/MaturityStaking.sol
function _getPendingReward(address account) private view returns
(uint256) {
    // ...

    // Pending reward = (UserAmount * TotalAccumulated) - UserDebt
    uint256 accumulatedReward = (userStake.amount * currentAccReward)
    / 1e18;

    return accumulatedReward > userStake.rewardDebt
        ? accumulatedReward - userStake.rewardDebt
        : 0;
}
```

IMPACT:

This mismatch can mislead maintainers and reviewers, increasing risk of misunderstanding in future modifications.

RECOMMENDATIONS:

Update the comment to accurately reflect current variable naming.

```
function _getPendingReward(address account) private view returns
(uint256) {
```

```
        // ...  
-        // Pending reward = (UserAmount * TotalAccumulated) - UserDebt  
+        // Pending reward = (UserAmount * currentAccReward) - UserDebt  
        uint256 accumulatedReward = (userStake.amount * currentAccReward)  
        / 1e18;  
  
        return accumulatedReward > userStake.rewardDebt  
            ? accumulatedReward - userStake.rewardDebt  
            : 0;  
    }
```

4. CONCLUSION

In this audit, we thoroughly analyzed **AquaFlux** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cve.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec